

Jupyter Notebook Execution Report

Name: Your Name

Date: August 09, 2026

Cell 1: ■ Markdown

Hospital Readmission Risk — Complete ML Pipeline

****Single notebook**** covering the full machine-learning lifecycle for 30-day hospital readmission prediction.

| Requirement | Value |

|-----|-----|

| Dataset source | [UCI Diabetes 130-US Hospitals
(1999–2008)](<https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008>) |

| Minimum rows | ****200,000**** |

| Target variable | `readmitted` (1 = readmitted within 30 days) |

| Models | Logistic Regression, Decision Tree, Random Forest, XGBoost, Neural Network |

****Run all cells top-to-bottom.**** Set `USER\_EMAIL` in `ml\_utils.py` before deployment cells.

Cell 2: ■ Code

```
%matplotlib inline

import json
import time
import warnings
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import plotly.io as pio
from IPython.display import Markdown, display
```

```
warnings.filterwarnings("ignore")
```

```
from ml_utils import (  
    DEFAULT_CSV,  
    DEFAULT_TARGET_ROWS,  
    MIN_DATASET_ROWS,  
    ML_LIFECYCLE_STAGES,  
    PIPELINE_STATE_PATH,  
    MODELS_DIR,  
    TARGET_COLUMN,  
    USER_EMAIL,  
    ensure_project_paths,  
    flask_app_context,  
    load_pipeline_state,  
    save_pipeline_state,  
    validate_dataset_rows,  
)
```

```
ensure_project_paths()
```

```
PIPELINE = {stage: "pending" for stage in ML_LIFECYCLE_STAGES}
```

```
display(pd.DataFrame({"Stage": ML_LIFECYCLE_STAGES, "Status": ["pending"] *  
len(ML_LIFECYCLE_STAGES)}))
```

Error:

Traceback (most recent call last):

```
File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line  
    exec('\n'.join(lines[:-1]), glb)  
File "<string>", line 1  
    %matplotlib inline  
    ^
```

SyntaxError: invalid syntax

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line  
    exec(source, glb)  
File "<string>", line 1  
    %matplotlib inline  
    ^
```

SyntaxError: invalid syntax

Cell 3: ■ Markdown

1. Problem Definition

Clinical problem: Predict whether a diabetic patient will be **readmitted within 30 days** after hospital discharge.

Business objective: Enable care teams to prioritise high-risk patients for follow-up and reduce avoidable readmissions.

ML task: Binary classification on structured EHR/tabular encounter data.

Success metrics: Accuracy, precision, recall, F1-score, ROC-AUC, PR-AUC; clinical utility via risk stratification (low / moderate / high).

Cell 4: ■ Code

```
PROBLEM = {  
    "target": TARGET_COLUMN,  
    "positive_class_meaning": "Readmitted within 30 days",  
    "negative_class_meaning": "Not readmitted within 30 days",  
    "minimum_dataset_rows": MIN_DATASET_ROWS,  
    "primary_metrics": ["accuracy", "precision", "recall", "f1_score", "roc_auc"],  
    "deployment_surface": "Flask web app – predictions, explainability, analytics",  
}  
  
display(pd.DataFrame([PROBLEM]).T.rename(columns={0: "value"}))  
  
PIPELINE["Problem Definition"] = "complete"
```

Error:

Traceback (most recent call last):

File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line
exec('\n'.join(lines[:-1]), glb)

File "<string>", line 2, in <module>

NameError: name 'TARGET_COLUMN' is not defined

Cell 5: ■ Markdown

2. Data Acquisition

Download the UCI Diabetes 130-US Hospitals dataset, clean encounters, engineer features, and expand to **≥200,000 rows** (bootstrap + numeric perturbation from ~102k real records).

Cell 6: ■ Code

```
from scripts.prepare_diabetes_dataset import prepare_diabetes_csv

csv_path = prepare_diabetes_csv(target_rows=DEFAULT_TARGET_ROWS)
raw_df = pd.read_csv(csv_path)
row_count = validate_dataset_rows(raw_df)

print(f"Dataset: {csv_path}")
print(f"Rows: {row_count:,} | Columns: {raw_df.shape[1]}")
print(f"Readmission rate: {raw_df[TARGET_COLUMN].mean():.1%}")
display(raw_df.head())

PIPELINE["Data Acquisition"] = "complete"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'scripts'
```

Cell 7: ■ Markdown

3. Data Understanding

Profile schema, dtypes, missing values, and basic target prevalence on the full acquired dataset.

Cell 8: ■ Code

```
overview = pd.DataFrame({
    "dtype": raw_df.dtypes.astype(str),
    "non_null": raw_df.count(),
    "missing": raw_df.isnull().sum(),
    "unique": raw_df.nunique(),
})
display(overview)
display(raw_df.describe(include="all").T.head(20))

PIPELINE["Data Understanding"] = "complete"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
NameError: name 'pd' is not defined
```

Cell 9: ■ Markdown

4. Data Cleaning & Preprocessing

- Impute missing numeric values (mean) and categoricals (`Unknown`)
- Label-encode categorical features
- Standard-scale numeric features
- IQR outlier detection (report only)
- Register dataset in the application database

Cell 10: ■ Code

```
import shutil

from app.extensions import db
from app.models.dataset import Dataset
from app.models.user import User
from app.services.dataset_service import analyze_csv
from app.services.preprocessing_service import PreprocessingConfig,
run_preprocessing_pipeline

with flask_app_context() as app:
    user = User.query.filter_by(email=USER_EMAIL).first()
    if user is None:
        raise RuntimeError(f"Register {USER_EMAIL} at /auth/register first.")

    csv_path = Path(DEFAULT_CSV)
    dataset = Dataset.query.filter_by(user_id=user.id,
original_filename=csv_path.name).first()
    upload_folder = Path(app.config["UPLOAD_FOLDER"])

    if dataset is None:
        stored = f"user{user.id}_{int(time.time())}_{csv_path.name}"
```

```

dest = upload_folder / stored
shutil.copy2(csv_path, dest)
analysis = analyze_csv(dest)
dataset = Dataset(
    user_id=user.id,
    original_filename=csv_path.name,
    stored_filename=stored,
    file_path=str(dest),
    file_size=dest.stat().st_size,
    row_count=analysis["row_count"],
    column_count=analysis["column_count"],
    missing_values=analysis["missing_values"],
    duplicate_rows=analysis["duplicate_rows"],
)
dataset.feature_names = analysis["feature_names"]
dataset.missing_by_column = analysis["missing_by_column"]
db.session.add(dataset)
db.session.commit()
else:
    shutil.copy2(csv_path, Path(dataset.file_path))
    analysis = analyze_csv(Path(dataset.file_path))
    dataset.row_count = analysis["row_count"]
    db.session.commit()

```

```

validate_dataset_rows(dataset.row_count)
USE_SMOTE = dataset.row_count < 250_000
preprocess_config = PreprocessingConfig(
    target_column=TARGET_COLUMN,
    missing_strategy="mean",
    categorical_encoding="label",
    scaling_method="standard",
    outlier_method="iqr",
    outlier_action="report_only",
    test_size=0.2,
    apply_smote=USE_SMOTE,

```

```

random_state=42,
)
processed = run_preprocessing_pipeline(
dataset, preprocess_config, Path(app.config["PROCESSED_FOLDER"]))
)
PREPROCESS_REPORT = json.loads(processed.report_json)
DATASET_ID = dataset.id
PROCESSED_ID = processed.id
USER_ID = user.id
TRAIN_PATH = processed.train_file_path
TEST_PATH = processed.test_file_path
SMOTE_PATH = processed.train_smote_file_path
TRAIN_ROWS = processed.train_rows
TEST_ROWS = processed.test_rows
SMOTE_ROWS = processed.train_smote_rows

print("Preprocessing report:", list(PREPROCESS_REPORT.keys()))
PIPELINE["Data Cleaning & Preprocessing"] = "complete"

```

Error:

```

Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 10, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 3, in <module>
ModuleNotFoundError: No module named 'app'

```

Cell 11: ■ Markdown

5. Exploratory Data Analysis (EDA)

Visualise missing values, class balance, correlations, readmission rates by category, and numeric summaries (10k sample for chart performance).

Cell 12: ■ Code

```

from app.services.eda_service import build_eda_charts

eda_sample = raw_df.sample(min(10_000, len(raw_df)), random_state=42)
eda_sample.to_csv("/tmp/eda_sample.csv", index=False)

```

```
eda_result = build_eda_charts(Path("/tmp/eda_sample.csv"))

for name, figure in eda_result["charts"].items():
    if figure:
        print("----", name, "----")
        pio.show(figure, renderer="notebook_connected")

numeric = raw_df.select_dtypes("number")
display(numeric.corr()[TARGET_COLUMN].sort_values(ascending=False).head(12).to_frame("correlation"))

PIPELINE["Exploratory Data Analysis (EDA)"] = "complete"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'app'
```

Cell 13: ■ Markdown

6. Feature Engineering

Engineered during acquisition (see `scripts/prepare\_diabetes\_dataset.py`):

- `admission\_source`, `medical\_specialty`, `payer\_code` (grouped top categories)
- `primary\_diagnosis\_group`, `secondary\_diagnosis\_group` (ICD grouping)
- `total\_prior\_visits` = outpatient + emergency + inpatient visits
- `lab\_procedures`, `diagnosis\_count` from UCI clinical counts

Cell 14: ■ Code

```
engineered = [
    "admission_source", "medical_specialty", "payer_code",
    "primary_diagnosis_group", "secondary_diagnosis_group",
    "total_prior_visits", "lab_procedures", "diagnosis_count",
]

present = [c for c in engineered if c in raw_df.columns]
display(raw_df[present].head())
```

```
print(f"Engineered features present: {len(present)}")
PIPELINE["Feature Engineering"] = "complete"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 6, in <module>
NameError: name 'raw_df' is not defined
```

Cell 15: ■ Markdown

7. Feature Selection

Remove highly correlated features (>0.95) and rank remaining features with ANOVA F-score ('SelectKBest').

Cell 16: ■ Code

```
from sklearn.feature_selection import SelectKBest, f_classif

train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)
feature_cols = [c for c in train_df.columns if c != TARGET_COLUMN]
X_train_full = train_df[feature_cols]
y_train = train_df[TARGET_COLUMN]

corr = X_train_full.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop_corr = [col for col in upper.columns if any(upper[col] > 0.95)]
X_train_reduced = X_train_full.drop(columns=drop_corr, errors="ignore")

k = min(20, X_train_reduced.shape[1])
selector = SelectKBest(score_func=f_classif, k=k)
selector.fit(X_train_reduced, y_train)
SELECTED_FEATURES = X_train_reduced.columns[selector.get_support()].tolist()

scores = pd.DataFrame({
    "feature": X_train_reduced.columns,
    "f_score": selector.scores_,
```

```

"selected": selector.get_support(),
}).sort_values("f_score", ascending=False)
display(scores.head(25))
print("Dropped (correlation):", drop_corr)
print("Selected features:", SELECTED_FEATURES)
PIPELINE["Feature Selection"] = "complete"

```

Error:

```

Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 3, in <module>
NameError: name 'pd' is not defined

```

Cell 17: ■ Markdown

8. Data Splitting

80/20 stratified train/test split. SMOTE applied on training set when dataset < 250k rows.

Cell 18: ■ Code

```

split_summary = {
    "train_rows": TRAIN_ROWS,
    "test_rows": TEST_ROWS,
    "train_rate": round(train_df[TARGET_COLUMN].mean(), 4),
    "test_rate": round(test_df[TARGET_COLUMN].mean(), 4),
    "smote_applied": USE_SMOTE,
    "smote_train_rows": SMOTE_ROWS,
}
display(pd.DataFrame([split_summary]))
if SMOTE_PATH:
    smote_df = pd.read_csv(SMOTE_PATH)
    print("SMOTE train readmission rate:", round(smote_df[TARGET_COLUMN].mean(), 4))
PIPELINE["Data Splitting"] = "complete"

```

Error:

```

Traceback (most recent call last):

```

```
File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
File "<string>", line 2, in <module>
NameError: name 'TRAIN_ROWS' is not defined
```

Cell 19: ■ Markdown

9. Model Selection

Model	Rationale
Logistic Regression	Interpretable linear baseline
Decision Tree	Simple non-linear rules
Random Forest	Robust ensemble for tabular healthcare data
XGBoost	State-of-the-art gradient boosting
Neural Network	Non-linear pattern learning

Cell 20: ■ Code

```
from app.ml.model_registry import MODEL_REGISTRY, get_model_label

selection = pd.DataFrame([
    {"key": k, "label": get_model_label(k), "description": v["description"]}
    for k, v in MODEL_REGISTRY.items()
])

display(selection)

PIPELINE["Model Selection"] = "complete"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'app'
```

Cell 21: ■ Markdown

10. Model Training

Train all five registered models on the preprocessed data (SMOTE train set when available). Artefacts sync to the Flask app database.

Cell 22: ■ Code

```
from app.ml.model_registry import serializable_parameters
from app.ml.training import create_training_record, train_model_sync
from app.models.processed_dataset import ProcessedDataset
from app.models.trained_model import TrainedModel

with flask_app_context() as app:
    from app.extensions import db
    processed = db.session.get(ProcessedDataset, PROCESSED_ID)
    use_smote = processed.train_smote_file_path is not None
    models_folder = Path(app.config["MODELS_FOLDER"])
    training_results = []

    for model_type in MODEL_REGISTRY:
        label = get_model_label(model_type)
        print(f"Training {label}...")
        trained = create_training_record(
            user_id=USER_ID,
            processed_dataset_id=PROCESSED_ID,
            model_type=model_type,
            parameters=serializable_parameters(model_type, {}),
            use_smote=use_smote,
        )
        start = time.perf_counter()
        train_model_sync(trained.id, models_folder, use_smote=use_smote)
        elapsed = time.perf_counter() - start
        metrics = trained.metrics or {}
        joblib.dump(
            joblib.load(trained.model_file_path),
            MODELS_DIR / f"notebook_{model_type}.joblib",
        )
```

```
training_results.append({
    "model": model_type,
    "id": trained.id,
    "accuracy": metrics.get("accuracy"),
    "f1": metrics.get("f1_score"),
    "roc_auc": metrics.get("roc_auc"),
    "seconds": round(elapsed, 1),
})
```

```
TRAINING_DF = pd.DataFrame(training_results)
display(TRAINING_DF)
```

```
PIPELINE["Model Training"] = "complete"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'app'
```

Cell 23: ■ Markdown

11. Hyperparameter Tuning

Randomized search (3-fold CV) on a 40k stratified subsample for **XGBoost** and **Random Forest**, then refit the best estimator on the full training set.

Cell 24: ■ Code

```
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from scipy.stats import randint, uniform
from app.ml.model_registry import build_estimator
from app.ml.evaluation import evaluate_model

train_path = SMOTE_PATH or TRAIN_PATH
tune_df = pd.read_csv(train_path)
if len(tune_df) > 40_000:
    tune_df = tune_df.sample(40_000, random_state=42)
```

```

X_tune = tune_df[SELECTED_FEATURES]
y_tune = tune_df[TARGET_COLUMN]

pos = int((y_tune == 1).sum())
neg = int((y_tune == 0).sum())
scale_pos_weight = neg / max(pos, 1)

xgb = build_estimator("xgboost", {"scale_pos_weight": scale_pos_weight})
xgb_search = RandomizedSearchCV(
    xgb,
    param_distributions={
        "n_estimators": randint(100, 400),
        "max_depth": randint(4, 12),
        "learning_rate": uniform(0.01, 0.15),
        "subsample": uniform(0.7, 0.3),
    },
    n_iter=12,
    cv=StratifiedKFold(3, shuffle=True, random_state=42),
    scoring="f1",
    n_jobs=-1,
    random_state=42,
    verbose=1,
)
xgb_search.fit(X_tune, y_tune)
print("Best XGBoost params:", xgb_search.best_params_)
print("Best CV F1:", round(xgb_search.best_score_, 4))

full_train = pd.read_csv(train_path)
X_full = full_train[SELECTED_FEATURES]
y_full = full_train[TARGET_COLUMN]
TUNED_MODEL = build_estimator("xgboost", {**xgb_search.best_params_,
    "scale_pos_weight": scale_pos_weight})
tune_start = time.perf_counter()
TUNED_MODEL.fit(X_full, y_full)
tune_time = time.perf_counter() - tune_start

X_test = test_df[SELECTED_FEATURES]

```

```

y_test = test_df[TARGET_COLUMN]

TUNED_EVAL = evaluate_model(TUNED_MODEL, X_test, y_test, tune_time,
                             feature_names=SELECTED_FEATURES)

display(pd.DataFrame([TUNED_EVAL["metrics"]]))

joblib.dump(
{
    "model": TUNED_MODEL,
    "model_type": "xgboost_tuned",
    "feature_columns": SELECTED_FEATURES,
    "target_column": TARGET_COLUMN,
    "parameters": xgb_search.best_params_,
    "metrics": TUNED_EVAL["metrics"],
},
MODELS_DIR / "notebook_xgboost_tuned.joblib",
)

PIPELINE["Hyperparameter Tuning"] = "complete"

```

Error:

```

Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 3, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 3, in <module>
ModuleNotFoundError: No module named 'app'

```

Cell 25: ■ Markdown

12. Model Evaluation

Compare all trained models plus the tuned XGBoost on hold-out test metrics and diagnostic charts.

Cell 26: ■ Code

```

from app.ml.comparison import build_model_comparison

with flask_app_context():
    comparison = build_model_comparison(USER_ID)
    BEST_MODEL_ID = comparison["best_model_id"]
    if BEST_MODEL_ID is None and comparison["models"]:

```

```

BEST_MODEL_ID = max(comparison["models"], key=lambda row: row.get("f1_score") or 0)["id"]
display(pd.DataFrame(comparison["models"]))

for name, figure in comparison["charts"].items():
    if figure:
        print("---", name, "---")
        pio.show(figure, renderer="notebook_connected")

print("Tuned XGBoost test F1:", TUNED_EVAL["metrics"]["f1_score"])
for name, figure in TUNED_EVAL["charts"].items():
    if figure:
        print("--- tuned:", name, "---")
        pio.show(figure, renderer="notebook_connected")

PIPELINE["Model Evaluation"] = "complete"

```

Error:

```

Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'app'

```

Cell 27: ■ Markdown

13. Model Interpretation

Global and local SHAP/LIME explanations for the best baseline model from the registry.

Cell 28: ■ Code

```

from app.explainability.explanation_service import build_full_explanations, list_explainable_models
from app.explainability.shap_service import build_global_explanations

with flask_app_context():
    explainable = list_explainable_models(USER_ID)
    interpret_model_id = BEST_MODEL_ID or (explainable[0].id if explainable else None)
    if interpret_model_id is None:

```

```

raise RuntimeError("No explainable models available.")
try:
    explanation = build_full_explanations(USER_ID, interpret_model_id,
    test_row_index=0)
except Exception as exc:
    print(f"Full SHAP/LIME failed ({exc}); falling back to global SHAP only.")
    explanation = build_global_explanations(USER_ID, interpret_model_id)
    for name, figure in explanation.get("charts", {}).items():
        if figure:
            print("----", name, "----")
            pio.show(figure, renderer="notebook_connected")

if TUNED_EVAL.get("charts", {}).get("feature_importance"):
    print("---- tuned model feature importance ----")
    pio.show(TUNED_EVAL["charts"]["feature_importance"], renderer="notebook_connected")

PIPELINE["Model Interpretation"] = "complete"

```

Error:

```

Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'app'

```

Cell 29: ■ Markdown

14. Model Deployment

Verify artefacts, run a prediction smoke test, and persist pipeline state for monitoring. Models are available in the Flask app at `/models/`, `/predictions/`, `/explainability/`.

Cell 30: ■ Code

```

from app.services.prediction_service import predict_patient

with flask_app_context():
    trained = TrainedModel.query.filter_by(id=BEST_MODEL_ID, user_id=USER_ID).first()
    patient_row = test_df.drop(columns=[TARGET_COLUMN]).iloc[0].to_dict()
    prediction = predict_patient(USER_ID, BEST_MODEL_ID, patient_row)

```

```
display(pd.DataFrame([prediction]))

state = {
    "dataset_id": DATASET_ID,
    "processed_id": PROCESSED_ID,
    "best_model_id": BEST_MODEL_ID,
    "tuned_model_path": str(MODELS_DIR / "notebook_xgboost_tuned.joblib"),
    "selected_features": SELECTED_FEATURES,
    "row_count": int(row_count),
    "metrics": TUNED_EVAL["metrics"],
    "trained_at": time.strftime("%Y-%m-%d %H:%M:%S"),
}

from ml_utils import PIPELINE_STATE_PATH
save_pipeline_state(state)
print("Pipeline state saved:", PIPELINE_STATE_PATH)
print("Deploy: python run.py → http://127.0.0.1:5000")
PIPELINE["Model Deployment"] = "complete"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'app'
```

Cell 31: ■ Markdown

15. Monitoring & Maintenance

Track prediction volume, risk distribution, and model performance over time via the analytics dashboard and prediction history.

Cell 32: ■ Code

```
from app.models.prediction_record import PredictionRecord
from app.services.analytics_service import build_analytics_dashboard

MONITORING_THRESHOLDS = {
    "min_accuracy": 0.65,
```

```

"min_f1": 0.70,
"max_high_risk_share": 0.35,
"review_predictions_every_days": 30,
}

with flask_app_context():
    pred_count = PredictionRecord.query.filter_by(user_id=USER_ID).count()
    dashboard = build_analytics_dashboard(USER_ID)
    print("Logged predictions:", pred_count)
    print("Monitoring thresholds:", MONITORING_THRESHOLDS)
    for name, figure in dashboard.get("charts", {}).items():
        if figure:
            print("---", name, "---")
            pio.show(figure, renderer="notebook_connected")

prior = load_pipeline_state()
if prior.get("metrics"):
    print("Prior tuned F1:", prior["metrics"].get("f1_score"))
PIPELINE["Monitoring & Maintenance"] = "complete"

```

Error:

```

Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 1, in <module>
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'app'

```

Cell 33: ■ Markdown

16. Model Retraining

****Retrain when:****

- Hold-out F1 drops below monitoring threshold
- New UCI data refresh or expanded cohort (re-run acquisition)
- Scheduled review (e.g. every 30 days)

Re-execute acquisition → preprocessing → training cells, or run ``python scripts/train_all_models.py --email YOUR_EMAIL``.

Cell 34: ■ Code

```
RETRAIN_TRIGGERS = pd.DataFrame([
{"trigger": "F1 below threshold", "action": "Re-run sections 2-12"},
{"trigger": "New hospital data", "action": "Re-run prepare_diabetes_csv with
updated target_rows"},
{"trigger": "Scheduled maintenance", "action": "Monthly retrain + compare metrics
to pipeline_state.json"},
])
display(RETRAIN_TRIGGERS)

display(pd.DataFrame({"Stage": list(PIPELINE.keys()), "Status":
list(PIPELINE.values())}))

completed = sum(1 for v in PIPELINE.values() if v == "complete")
print(f"Pipeline complete: {completed}/{len(PIPELINE)} stages")
PIPELINE["Model Retraining"] = "documented"
```

Error:

```
Traceback (most recent call last):
  File "/home/schrodinger/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
NameError: name 'pd' is not defined
```